

Zerg Chess Project

Intelligent Search and games

Paola Pinna i6446765

Game Rules

Board: 8×5 grid

Asymmetry:

- **White:** Standard pieces (King, Rook, Bishop, Knight, Pawns)
- **Black:** Only pawns in 3 rows (“Zerg swarm”)

Win Conditions:

- **White wins** → All black pawns eliminated
- **Black wins** → A pawn reaches the last rank or all white pawns eliminated

No Check/Checkmate:

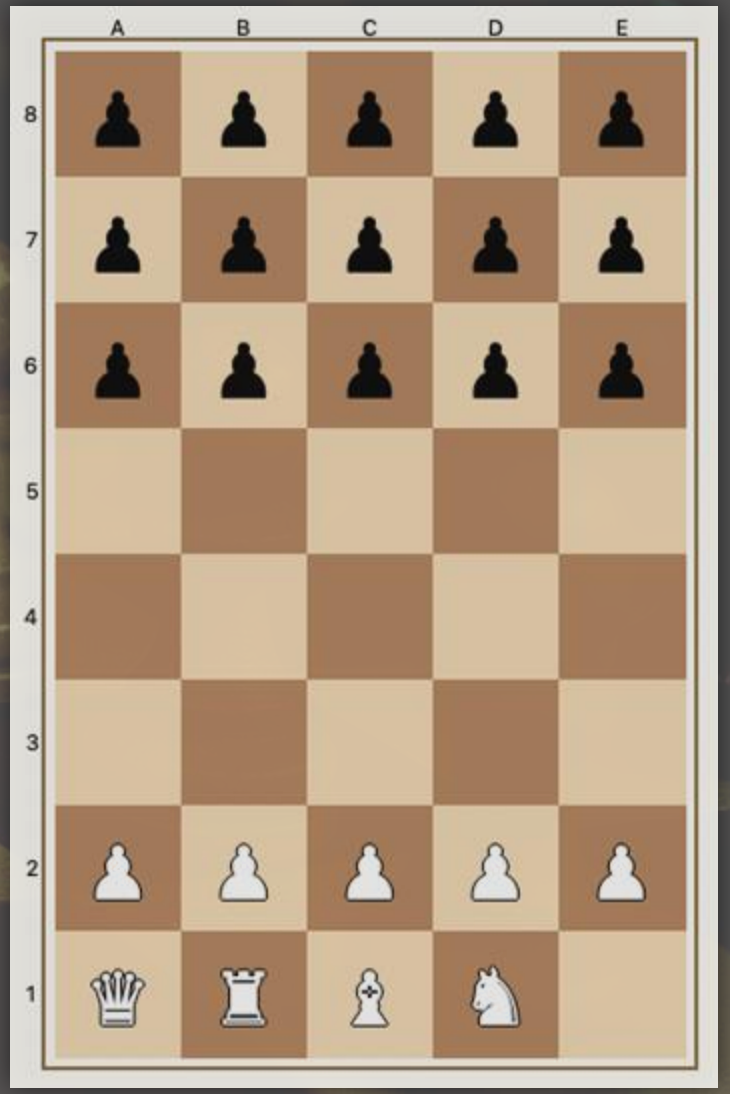
- Pure objective-based victory

Promotion Rule

- Only **White pawns** can promote (into a random piece in real gameplay, deterministic for AI)

En Passant Rule

- White pawn → can capture a pawn that advanced 2 squares



Engine Architecture & Modules

Core Game

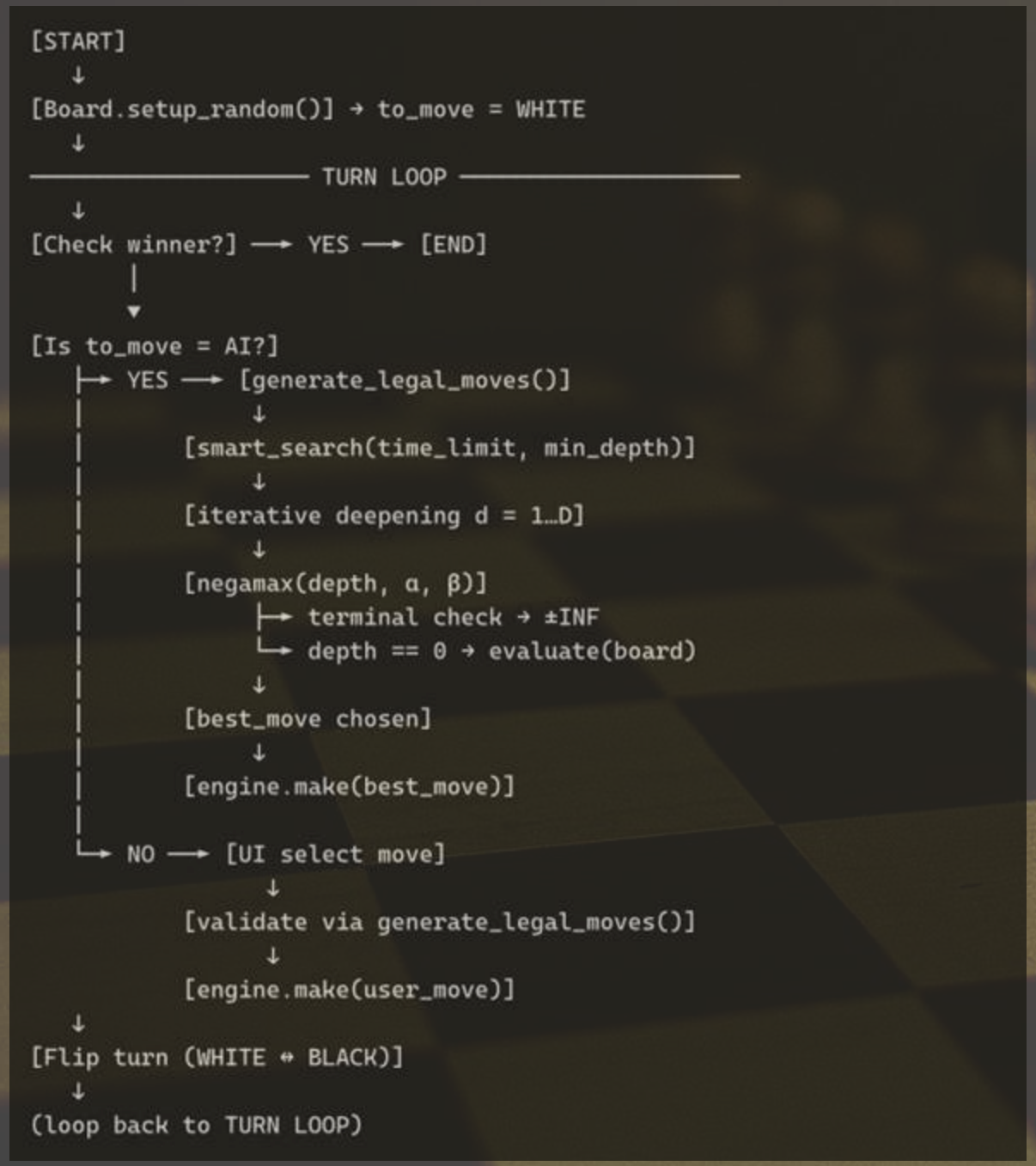
- **board.py** → Base structures (Grid, Move, Color, piece constants)
- **game.py** → Board class: grid, winner check, random setup
- **movegen.py** → Generates legal moves for each piece type
- **engine.py** → Make/undo moves, handles pawn promotion

UI

- **main.py** → launches UI
- **ui_tk.py** → Tkinter GUI: 8×5 board, click moves, AI vs AI mode

AI

- **evaluation.py** → Evaluates position: material, advanced pawns, mobility, square control.
- **negamax.py** → Best move search: alpha-beta pruning, TT, killer move, HH, iterative deepening, adaptive search.



Flow Diagram

Evaluation Principles

Goal of Evaluation

- Estimate how favorable a position is for White (negamax: positive = good for White, negative = good for Black).
- **Heuristics must reflect the difference in behaviors.**

Scores in this game are asymmetric to reflect different roles:

White ≈ Defender / Killer

Black ≈ Invader / Breaker

All values were manually tuned through AI self-play to avoid passive/stalemate and wrong behavior

Total Score =

+ (Material & Activity)

+ (Black Leader Threat)

+ (Pawn Coordination / Captures)

+ Mobility + Compactness

+ Anti-Stalemate

Scoring Strategy (AI Heuristic)

Code Components	Purpose in Game Logic
_eval_material_and_activity	Reward strong pieces & active white pawns (block, capture, promote)
_eval_black_leader	Encourage black aggression, the most advanced Black Pawn (existential threat)
_eval_black_pawn_coordination	Reward black for advancing pawns together (harder to break, group threat)
_eval_captured_black_pawns _eval_captured_white_pieces	White rewarded for killing BP / Black rewarded for killing white pieces
_eval_mobility	Favor side with more legal moves (space & flexibility)
_eval_compactness	Penalize scattered white structure, reward defensive shape
_eval_anti_stalemate	Prevents the AI from drifting into deadlocked positions where no progress is possible, especially for Black (forces aggression
Terminal Check (_eval_terminal)	Immediate win/loss detection and reward

Move Ordering

Even the best evaluation function is useless if the search explores moves in the wrong order.

- Reduces the search tree dramatically
- Increases alpha-beta pruning efficiency
- Allows deeper search with the same time limit

Ordering Pipeline in order_moves()

1. **Transposition Table Move**
2. **Killer Moves**
3. **Captures / Tactical (MVV-LVA + Leader threats)**
4. **Quiet Moves (History Heuristics)**
5. **Fallback / Unsorted leftovers**

Transposition Table

```
// Lookup before search
hash = compute_hash(position)
if TT has entry AND entry.depth ≥ current_depth:
    use entry.best_move at top of move list
```

Killer Moves

```
FOR the current search depth:
    IF a quiet move previously caused a cutoff:
        store it as a “killer move”
During move ordering:
    IF killer move is legal:
        prioritize it before other quiet moves
```

History Heuristic

```
Each quiet move tracks a "success score"
After search:
    IF quiet move improved alpha:
        increase its history score
During ordering:
    sort remaining quiet moves by their history score
```

Negamax Algorithm

```
function NEGAMAX(position, depth, α, β):
    if depth == 0 or game is over:
        return evaluate(position)
    best = -∞
    for move in ordered_moves(position):
        make(move)
        score = -NEGAMAX(new_position, depth - 1, -β, -α)
        undo(move)
        best = max(best, score)
        α = max(α, best)
        if α ≥ β:
            break   // prune branch
    return best
```

The AI searches moves from the current player's perspective, always maximizing its own score (by negating the opponent's value).

- Single function for both players (Minimax = Negamax with sign flip)
- Each recursive call inverts the score (score = -child_score)
- Alpha-Beta pruning removes hopeless branches early

Depth Search Strategy

The engine searches dynamically: it goes deeper when danger increases, but stays efficient using α–β pruning, iterative deepening, and time control.

PARAMETERS

- Max depth = 6
- Global Time limit per move = 1.0s “slow”, 20.5s “normal”, 0.2s “fast”.

ITERATIVE DEEPENING

- Starts from depth 1 → increases until time limit
- Returns best move found so far (safe under time pressure)

ADAPTIVE DEPTH SEARCH

Condition	Depth Adjustment
Few legal moves	Deeper search (positional game)
Many legal moves	Shallower search (fast wide search)
Black pawn leader advanced (r6–r7)	+1 or +2 depth (danger tracking)

Results Discussion

Key Insights

- **Dramatic Node Reduction:**
  Alpha-Beta pruning combined with move ordering reduces explored nodes by **90%** at higher depths, enabling deeper strategic searches within the same time.
- **Iterative Deepening + TT:**
  Iterative deepening improves move ordering at each iteration. Transposition Table hits (15–20%)
- **Killer Moves (2–3 per depth) → excellent early cutoffs**, effective against similar failing moves at the same depth.
- **Scalability:**
  Without pruning, depth 5 would be computationally infeasible (>400k nodes). With optimizations, deep searches become real-time playable.
- **Move ordering** is optimal
- **History Heuristic** → works in a moderate but consistent way

Outcome	Percentage
White Wins	**60%**
Black Wins	**35%**
Stalemate	**5%**

The engine plays **consistent and strategic games**, with White showing a natural initiative advantage